

Report:

The approach for my code was to use recursive logic to complete the Towers of Hanoi problem. The main function “hanoi” is using the general registers r1, r2, and r2 to keep track of the source, destination, and auxiliary pegs respectively with r4 as temporary storage. The function checks the base case (no disks left to move) and performs recursion to move n-1 disks, handling the moving of the nth disk. The stack pointer “sp” is vital in the recursion process. Since ARM doesn’t automatically take care of call stacks for recursive functions, it is important to manually save and restore the state of the program. The stack stores the return address “lr” and the registers before making each call, which lets each function call be preserved and restored when it completes.

Code:

The screenshot displays an ARMv7 simulator interface. On the left, the 'Registers' panel shows the state of registers r0 through r15, sp, lr, cpsr, and spsr. The 'Editor (Ctrl-E)' panel on the right contains the assembly code for a Towers of Hanoi solution. The code includes a recursive function 'hanoi' that uses registers r0, r1, r2, r3, and r4 to manage the recursive process. The 'Messages' panel at the bottom shows several breakpoint requests from the simulator, indicating that the program has reached specific instructions during its execution.

```
1 .text
2 .global _start
3
4 hanoi:
5     CMP r0, #0          // Compare the disks with 0
6     BEQ finish          // If disks is equal to 0, branch to finish to end the recursion
7     SUB r0, r0, #1      // Decrement the number of disks for the recursive call
8     PUSH {r1-r3, lr}    // Save the current states for source, destination, auxiliary pegs, and return address
9     MOV r4, r2           // Temporarily store the current destination in r4
10    MOV r2, r3           // Set the new destination to the auxiliary peg
11    MOV r3, r4           // Set the new auxiliary to the original destination
12    BL hanoi             // Recursive call to move n-1 disks from source to auxiliary using destination as temporary storage
13    MOV r4, r1           // Temporarily store the current source in r4
14    MOV r1, r3           // Set the new source to the auxiliary peg
15    MOV r3, r4           // Set the new auxiliary to the original source
16    BL hanoi             // Recursive call to move n-1 disks from new source to destination using original source as temporary storage
17    POP {r1-r3, lr}     // Restore the state of source, destination, auxiliary pegs, and return address
18    BX lr               // Return from the function
19
20 finish:
21     MOV pc, lr          // Set the program counter to the link register to return from the function
22
23 _start:
24     LDR sp, =stack_loc  // Load the address of the stack location into the stack pointer
25     MOV r0, #4          // Initialize r0 with the total number of disks
26     MOV r1, #1          // Initialize r1 with the identifier for the source peg
27     MOV r2, #2          // Initialize r2 with destination peg
28     MOV r3, #3          // Initialize r3 with auxiliary peg
29     BL hanoi             // Begin the recursion for the Tower of Hanoi
30     B _start            // After ending the hanoi process, loop back to _start (infinite loop)
31
32 .data
33 stack_loc: .word 0x20001000 // Define the initial main stack pointer value
34
35 .end
```

Messages:

- Simulator requested a breakpoint.
- At pc=00000038: Instruction opcode differs from original executable. Was e1a0f00e (mov pc, lr).now 00000003 (andeq r0, r0, r3). [Details...](#)
- Simulator requested a breakpoint.
- At pc=0000003c: Instruction opcode differs from original executable. Was e59fd014 (ldr sp, [pc, #20] ; 0x54).now 00000020 (andeq r0, r0, r0, LSR #32). [Details...](#)
- Simulator requested a breakpoint.
- At pc=00000048: Instruction opcode differs from original executable. Was e3a00004 (mov r0, #4 ; 0x4).now 00000001 (andeq r0, r0, r1). [Details...](#)
- Simulator requested a breakpoint.